

How does a Breadth-First Search work when looking for Shortest Path?

[Ask Question](#)

Asked 13 years, 9 months ago Modified 2 years, 3 months ago Viewed 265k times



183



I've done some research, and I seem to be missing one small part of this algorithm. I understand how a Breadth-First Search works, but I don't understand how exactly it will get me to a specific path, as opposed to just telling me where each individual node can go. I guess the easiest way to explain my confusion is to provide an example:

So for instance, let's say I have a graph like this:

A - B - C

| |

D - - - - - E

And my goal is to get from A to E (all edges are unweighted).

I start at A, because that's my origin. I queue A, followed by immediately dequeuing A and exploring it. This yields B and D, because A is connected to B and D. I thus queue both B and D.

I dequeue B and explore it, and find that it leads to A (already explored), and C, so I queue C. I then dequeue D, and find that it leads to E, my goal. I then dequeue C, and find that it also leads to E, my goal.

I know logically that the fastest path is A->D->E, but I'm not sure how exactly the breadth-first search helps - how should I be recording paths such that when I finish, I can analyze the results and see that the shortest path is A->D->E?

Also, note that I'm not actually using a tree, so there are no "parent" nodes, only children.

breadth-first-search

shortest-path

[Share](#) [Improve this question](#) [Follow](#)

edited Dec 19, 2022 at 10:18



[Antoine Cotten](#)

2,762 ● 21 ● 40

asked Dec 5, 2011 at 0:35



[Jake](#)

3,389 ● 4 ● 34 ● 52

- 3 "Also, note that I'm using not actually using a tree, so there are no "parent" nodes, only children" - well you obviously will have to store the parent somewhere. For DFS you're doing it indirectly through the call stack, for BFS you have to do it explicitly. Nothing you can do about it I fear :) – [Voo](#) Dec 5, 2011 at 0:50 

[Add a comment](#)

[Report this ad](#)

9 Answers

Sorted by: Highest score (default) 



118

Technically, Breadth-first search (BFS) by itself does not let you find the shortest path, simply because BFS is not looking for a shortest path: BFS describes a strategy for searching a graph, but it does not say that you must search for anything in particular.



[Dijkstra's algorithm](#) adapts BFS to let you find single-source shortest paths.



In order to retrieve the shortest path from the origin to a node, you need to maintain two items for each node in the graph: its current shortest distance, and the preceding node in the shortest path. Initially all distances are set to infinity, and all predecessors are set to empty. In your example, you set A's distance to zero, and then proceed with the BFS. On each step you check if you can improve the distance of a descendant, i.e. the distance from the origin to the predecessor plus the length of the edge that you are exploring is less than the current best distance for the node in question. If you can improve the distance, set the new shortest path, and remember the predecessor through which that path has been acquired. When the BFS queue is empty, pick a node (in your example, it's E) and traverse its predecessors back to the origin. This would give you the shortest path.

If this sounds a bit confusing, wikipedia has a nice [pseudocode section](#) on the topic.

[Share](#) [Improve this answer](#) [Follow](#)

edited Dec 1, 2014 at 21:34

answered Dec 5, 2011 at 0:52



[marzapower](#)

5,621 ● 7 ● 41 ● 77



[Sergey Kalinichenko](#)

729k ● 85 ● 1.2k ● 1.6k

- 75 I'd like to make the following note for people that look at this post in the future: If the edges are unweighted, there is no need to store the "current shortest distance" for each node. All that needs to be stored is the parent for each node discovered. Thus, while you are examining a node and enqueueing all of its successors, simply set the parent of those nodes to the node you are examining (this will double as marking them "discovered"). If this parent pointer is NUL/nil/None for any given node, it means either that it has not yet been discovered by BFS or it is the source/root node itself. – [Shashank](#) Sep 17, 2013 at 16:28

- 1 @Shashank If we are not maintaining distance then how would we know the shortest distance, please explain more. – [Gaurav Sehgal](#) Apr 12, 2015 at 19:17

- 17 @gauravsehgal That comment is for graphs with unweighted edges. BFS will find the shortest distance simply because of its radial-search pattern which considers nodes in order of their distance from the starting point. – [Shashank](#) Apr 12, 2015 at 21:48

22 A tip for readers of @Shashank's comment: unweighted and uniformly weighted (e.g. all edges have weight=5) are equivalent. – [TWiStErRob](#) Nov 30, 2015 at 12:23

3 It can be shown that, for unweighted graphs, the BFS is equivalent to Dijkstra's algorithm and thus finds the shortest path in this circumstance. – [Kyle](#) Nov 2, 2017 at 5:51

[Show 5 more comments](#)



76



As pointed above, BFS can **only** be used to find shortest path in a graph if:

1. There are no loops
2. All edges have same weight or no weight.

To find the shortest path, all you have to do is start from the source and perform a breadth first search **and stop when you find your destination Node**. The only additional thing you need to do is have an array previous[n] which will store the previous node for every node visited. The previous of source can be null.

To print the path, simple loop through the previous[] array from source till you reach destination and print the nodes. DFS can also be used to find the shortest path in a graph under similar conditions.

However, if the graph is more complex, containing weighted edges and loops, then we need a more sophisticated version of BFS, i.e. Dijkstra's algorithm.

Share Improve this answer Follow

edited Jul 22, 2015 at 12:55



[nbpro](#)

16k ● 34 ● 122 ● 217

answered Sep 13, 2013 at 5:07



[javaProgrammer](#)

1,225 ● 1 ● 11 ● 12

1 Dijkstra if no -ve weights else use bellman ford algo if -ve weights – [shaunak1111](#) Oct 17, 2013 at 7:06

Does BFS work to find all the shortest paths between two nodes? – [Maria Ines Parnisari](#) Oct 11, 2014 at 19:57

61 @javaProgrammer, it is not right. BFS can be used to find shortest path in an unweighted cyclic graph as well. If a graph is unweighted, then BFS can be applied for SP regardless of having loops. – [Andrei Kaigorodov](#) Jan 21, 2015 at 17:13

5 To print the path, simple loop through the previous[] array from source till you reach destination. But previous of source is null. I think what you meant is, **backtrace from destination using the previous array until you reach the source**. – [aandis](#) Jan 22, 2016 at 6:40

2 Why do you say DFS will work under similar conditions? Is it not possible for DFS to take a naive round-about route to get from nodes start->end, and therefore give you a path that is not the shortest? – [James Wierzba](#) Jun 23, 2016 at 21:02

[Show 3 more comments](#)



From [tutorial here](#)

33



"It has the extremely useful property that if all of the edges in a graph are unweighted (or the same weight) then the first time a node is visited is the shortest path to that node from the source node"



Share Improve this answer Follow

edited Dec 17, 2012 at 5:58

answered Dec 17, 2012 at 5:38



Himanshu

32.7k ● 32 ● 117 ● 137



acheron55

5,629 ● 2 ● 25 ● 23

This is good for directly reachable node (1->2) (2 is reached directly from 1). For non-directly reachable node there is more work (1->2->3, 3 is not reached directly from 1). Of course, it is still true considering individually, i.e. 1->2 & 2->3 individually are shortest paths. – [Manohar Reddy Poreddy](#) Sep 28, 2015 at 6:42

Add a comment



I have wasted 3 days
ultimately solved a graph question

17



used for
finding shortest distance
using BFS



Want to share the experience.

When the (undirected for me) graph has
fixed distance (1, 6, etc.) for edges

#1

We can use BFS to find shortest path simply by traversing it
then, if required, multiply with fixed distance (1, 6, etc.)

#2

As noted above
with BFS

the very 1st time an adjacent node is reached, it is shortest path

#3

It does not matter what queue you use
deque/queue(c++) or
your own queue implementation (in c language)
A circular queue is unnecessary

#4

Number of elements required for queue is N+1 at most, which I used
(dint check if N works)
here, N is V, number of vertices.

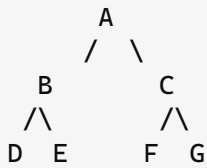
#5

Wikipedia BFS will work, and is sufficient.
https://en.wikipedia.org/wiki/Breadth-first_search#Pseudocode

I have lost 3 days trying all above alternatives, verifying & re-verifying again and again above they are not the issue.

(Try to spend time looking for other issues, if you don't find any issues with above 5).

More explanation from the comment below:



Assume above is your graph

graph goes downwards

For A, the adjacents are B & C

For B, the adjacents are D & E

For C, the adjacents are F & G

say, start node is A

1. when you reach A, to B & C the shortest distance to B & C from A is 1
2. when you reach D or E, thru B, the shortest distance to A & D is 2 (A->B->D)

similarly, A->E is 2 (A->B->E)

also, A->F & A->G is 2

So, now instead of 1 distance between nodes, if it is 6, then just multiply the answer by 6
example,

if distance between each is 1, then A->E is 2 (A->B->E = 1+1)

if distance between each is 6, then A->E is 12 (A->B->E = 6+6)

yes, bfs may take any path

but we are calculating for all paths

if you have to go from A to Z, then we travel all paths from A to an intermediate I, and since there will be many paths we discard all but shortest path till I, then continue with shortest path ahead to next node J

again if there are multiple paths from I to J, we only take shortest one

example,

assume,

A -> I we have distance 5

(STEP) assume, I -> J we have multiple paths, of distances 7 & 8, since 7 is shortest

we take A -> J as 5 (A->I shortest) + 8 (shortest now) = 13

so A->J is now 13

we repeat now above (STEP) for J -> K and so on, till we get to Z

Read this part, 2 or 3 times, and draw on paper, you will surely get what i am saying, best of luck

Share Improve this answer Follow

edited Jan 22, 2017 at 8:30

answered Sep 30, 2015 at 16:58



Manohar Reddy Poreddy

27.9k ● 10 ● 169 ● 150

- 1 Could you please elaborate how did you manage to find the shortest path with a breadth first search. A breadth first search mainly searches for a node, there can be n paths to a goal node from the source node & bfs may take any path. How are you determining the best path? – techPackets Jan 22, 2017 at 7:32

i have added 'more explanation' part to above answer, let me know if that satisfies

– Manohar Reddy Poreddy Jan 22, 2017 at 8:24

- 1 I see you are trying to run a BFS on weighted graph. Of distances 7 & 8 why did you choose 8? why not 7? what if the 8 node doesn't have further edges to destination? The flow will have to choose 7 then. – techPackets Jan 22, 2017 at 11:01 ✎

good question, upvoted, yes, we are not throwing away, we keep track of all adjacent nodes, until we reach destination. BFS will only work when there are only constant distances like all 7 or all 8. I gave a generic one which has 7 & 8 which is also called as dijkstra's algorithm. – Manohar Reddy Poreddy Jan 23, 2017 at 4:54 ✎

sorry, not sure what you mean, see en.wikipedia.org/wiki/Dijkstra's_algorithm

– Manohar Reddy Poreddy Jan 23, 2017 at 19:08 ✎

Add a comment



7

Breadth-first search will always find the shortest path in an unweighted graph (for weighted graphs, see Dijkstra's algorithm). The graph may be cyclic or acyclic. Pseudocode is below.



Note that the following pseudocode uses a queue to manage which vertices to visit. It also assumes you have vertex objects where each vertex is initialized with



```
vertex {
    visited = false
    distance = infinity
    name     = "anything unique" [or, "start_vertex"/"end_vertex"]
    children = [ ... ]
}
```

`start_vertex` below is a reference to the vertex with the name `start_vertex`

Algorithm:

```

start_vertex.distance = 0
queue.enqueue(start_vertex)

while not queue.empty():
    curr = queue.dequeue()

    if curr.name == "end_vertex":
        return curr.distance

    # deals with cyclic graphs
    if curr.visited:
        continue

    curr.visited = true
    curr_child_distance = curr.distance + 1

    for child in curr.children:
        if not child.visited and child.distance > curr_child_distance:
            child.distance = curr_child_distance
            queue.enqueue(child)

# no path connecting the vertices if we get here
return -1

```

Share Improve this answer Follow

edited Jun 1, 2023 at 21:50

answered Jul 28, 2020 at 21:33



Matt

1,176 ● 2 ● 12 ● 22

Add a comment



Based on [acheron55 answer](#) I posted a possible implementation [here](#).
Here is a brief summery of it:

3



All you have to do, is to keep track of the path through which the target has been reached. A simple way to do it, is to push into the `Queue` the whole path used to reach a node, rather than the node itself.



The benefit of doing so is that when the target has been reached the queue holds the path used to reach it.



This is also applicable to cyclic graphs, where a node can have more than one parent.

Share Improve this answer Follow

answered Jan 15, 2018 at 9:44



c0der

18.9k ● 6 ● 38 ● 70

Add a comment



I had the same lack of understanding. It took me a while to wrap my head around how BFS finds the shortest path between any two nodes.

0



Unfortunately, most answers are unhelpful and convolute the matter by bringing in Dijkstra's algorithm unnecessarily. I shall expand upon the answer by Himanshu who raised the crucial point that...



Just by the way BFS traverses a graph, any node it reaches is the shortest path to that node from the source node. Suppose we need to find the path D-C in the following simple graph. Follow along using pen and paper.

[Smallgraph4nodes](#)

We start from D and explore the graph using BFS. You see that it's certain that we shall reach C through B and not A because B would come first in our traversal. This is why we think of nodes as parents and children and why this information is stored in the form of a dictionary as we move along the graph for later retrieval to construct a path. Suppose this dict is called 'rel_dict', rel for relationship.

Let's make this graph bigger (picture and code added for the graph). Now, we'll see that to find the path between any two nodes, it's important that we start from one of the two nodes. Because if we choose a third node the rel_dict so formed at the end of the BFS traversal would possibly not give the shortest path but instead either no path at all or a longer one.

[7nodesgraph](#)

```
import networkx as nx
import matplotlib.pyplot as plt

d = {
    "A": ["B", "C"],
    "B": ["A", "D", "E", "C"],
    "C": ["A", "B", "F"],
    "D": ["B"],
    "E": ["B", "G"],
    "F": ["C"],
    "G": ["E"]
}

g = nx.Graph(d)
nx.draw_networkx(g)
plt.draw()
plt.show()
```

To dequeue a node we enqueue its children, if any. Simultaneously for each node visited we map it to its parent in the rel_dict dictionary. We keep doing this till the queue empties out. Then to construct the path between the starting node and any other node we, take the ending node find its parent then find the parent of this parent and so on, till we find the starting node and we will. Do this exercise by pen and paper: you'll have to maintain the queue and the rel_dict. Then construct the path in reverse.

Now, try to find the path between any two nodes neither of which were the starting nodes and you'll see that it doesn't give the correct answer.

Share Improve this answer Follow

answered Feb 13, 2023 at 7:32



Lifeisafartinthewind

71 ● 6

Add a comment



-1

A good explanation of how BFS computes shortest paths, accompanied by the most efficient simple BFS algorithm of which I'm aware and also by working code, is provided in the following peer-reviewed paper:



<https://queue.acm.org/detail.cfm?id=3424304>



The paper explains how BFS computes a shortest-paths tree represented by per-vertex parent pointers, and how to recover a particular shortest path between any two vertices from the parent pointers. The explanation of BFS takes three forms: prose, pseudocode, and a working C program.

The paper also describes "Efficient BFS" (E-BFS), a simple variant of classic textbook BFS that improves its efficiency. In the asymptotic analysis, running time improves from $\Theta(V+E)$ to $\Omega(V)$. In words: classic BFS always runs in time proportional to the number of vertices plus the number of edges, whereas E-BFS sometimes runs in time proportional to the number of vertices alone, which can be much smaller. In practice E-BFS can be much faster, depending on the input graph. E-BFS sometimes offers no advantage over classic BFS but it's never much slower.

Remarkably, despite its simplicity E-BFS appears not to be widely known.

Share Improve this answer Follow

edited Oct 28, 2020 at 18:25

answered Oct 28, 2020 at 18:20



Terence Kelly

1 ● 1

- 1 Hi Terrence, welcome to stack overflow. The link looks interesting but your answer itself doesn't seem to attempt to answer the question. As links can and do break, it's always appreciated if an answer tries to include relevant details from an linked resources. – Michael Welch Oct 29, 2020 at 0:40

Add a comment



-13



The following solution works for all the test cases.

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
```



```
public class Solution {

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        int testCases = sc.nextInt();

        for (int i = 0; i < testCases; i++)
        {
            int totalNodes = sc.nextInt();
            int totalEdges = sc.nextInt();

            Map<Integer, List<Integer>> adjacencyList = new HashMap<Integer>

            for (int j = 0; j < totalEdges; j++)
            {
                int src = sc.nextInt();
                int dest = sc.nextInt();

                if (adjacencyList.get(src) == null)
                {
                    List<Integer> neighbours = new ArrayList<Integer>();
                    neighbours.add(dest);
                    adjacencyList.put(src, neighbours);
                } else
                {

```

Share Improve this answer Follow

answered Jan 29, 2016 at 19:43



[user3819236](#)

189 ● 2 ● 10

11 Downvoted for not answering the question. Simply pasting a code snippet won't work on SO.
– [Rishabh Agrahari](#) Jul 25, 2017 at 8:02

Add a comment

Start asking to get answers

Find the answer to your question by asking.

[Ask question](#)

Explore related questions

[breadth-first-search](#)

[shortest-path](#)

See similar questions with these tags.